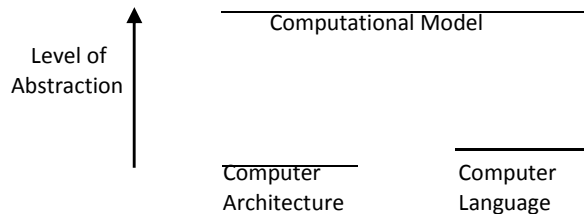


UNIT 1: Computational Models

The Concept of Computational Model :- The computer architecture and language classes must have a common foundation or paradigm called a **Computational Model**. The concept of a computational model represents a higher level of abstraction than either the computer architecture or the programming language alone, and covers both, as show below -

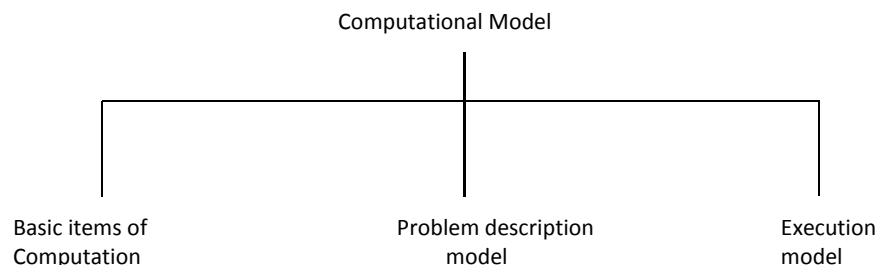


Interpretation of the computational model concept as a high-level abstraction.

The Concept of Computation Model - The concept of computational model comprises the set of the following three abstractions –

1. The basic items of computations
2. The problem description model
3. The execution model

Contrary to initial thoughts, the set of abstractions that should be chosen to specify computational models is far from obvious. A smaller number of criteria would define fewer but more basic computational models, while a larger number of criteria would result in a relatively large number of distinct models.



Interpretation of the computational model concept as covering three abstraction.

The first abstraction identifies the basic items of computation. This is a specification of the items the computation refers to and the kind of computations (operations) that can be performed on them.

For instance, in the von Neumann computational model the basic items of computation are data. This data will typically be represented by named (that is, indentifiable) entities in order to be able to distinguish between a number of different data items in the course of a computation. These named entities are commonly called variables in programming languages and are implemented by memory or register addresses in architectures.

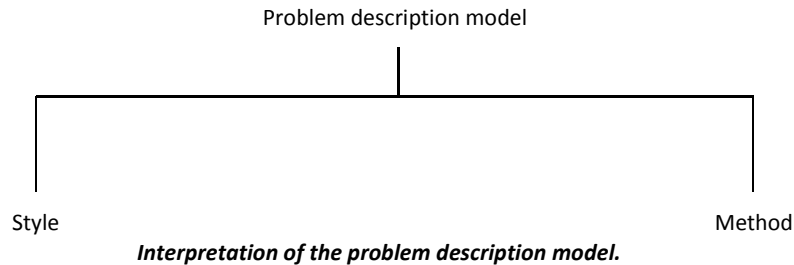
All the best known computational models, namely –

- the Turing model.
 - the von Neumann model
 - the dataflow model
- are based on the concept of data.

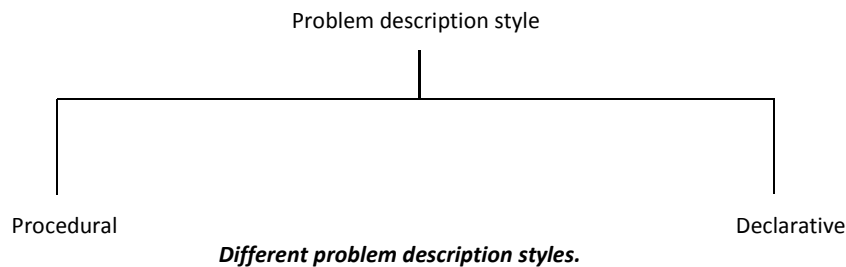
In contrast , there are a number of models which are not data based. In these models the basic items of computation are:

- Objects or messages sent to them requiring an associated manipulation (as in the object-based model)
- Arguments and the functions applied on them (applicative model)
- Elements of sets and the predicates declared on them (predicate-logic-based model)

The problem description model refers to both the style and method of problem description, as shown –



The problem description style specified how problems in a particular computational model are described. The style is either procedural or declarative, as shown –

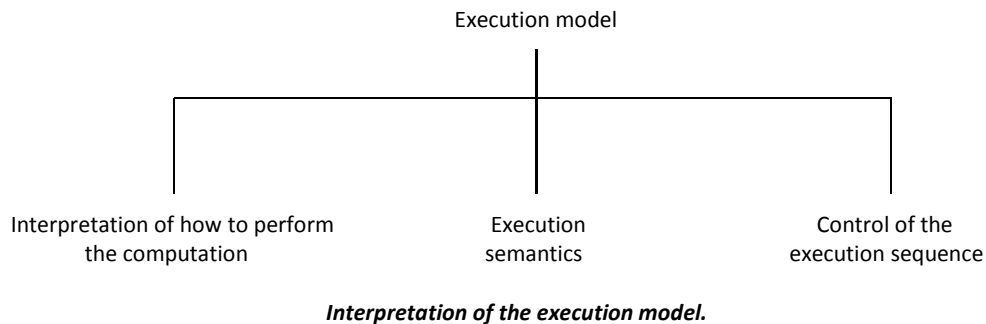


In a procedural style the algorithm for solving the problem is stated. A particular solution is then declared in the form of an algorithm.

In a declarative style all the facts and relationships relevant to the given problem have to be stated. There are two methods for expressing these relationships and facts. The first uses functions, as in the applicative model of computation, while the second states the relationships and facts in the form of predicates, as in the predicate-logic-based computational model.

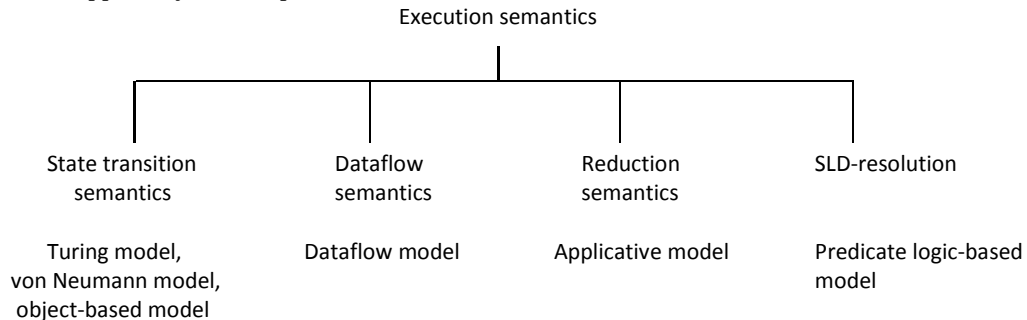
The other component of the problem description model is the problem description method. It is interpreted differently for the procedural and the declarative style. When using the procedural style the problem description model states how a solution of the given problem has to be described. In contrast, while using the declarative style, it specifies how the problem itself has to be described.

The third and the last element of computational model outlines the execution model. It consists of three components as shown –



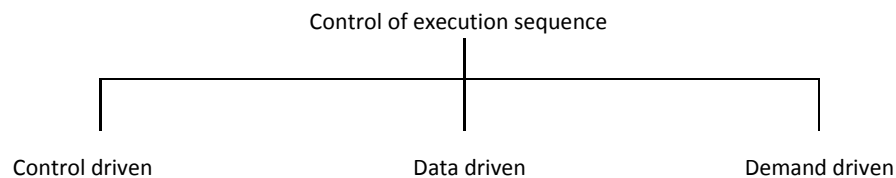
The first component declares the interpretation of the computation, which is strongly related to the problem description method. The choice of problem description method and the interpretation of the computation mutually determine and presume each other.

The next component of the execution model specifies the execution semantics. This can be interpreted as a rule that prescribes how a single execution step is to be performed. This rule is, of course, associated with the chosen problem description method and how the execution of the computation is interpreted. The different kinds of execution semantics applied by the computational models are summarized below –



Main types of execution semantics.

The last component of the model specifies the control of the execution sequences. In the basic models, execution is either control driven or data driven or demand driven as shown –



Basic ways of controlling the execution sequence.

For **Control-driven execution** it is assumed that there exists a program consisting of a sequence of instructions. The execution sequence is then implicitly given by the order of the instructions. However, explicit control instructions can also be used to specify a departure from the implied (varfuZfgr) execution sequence.

Data-driven execution is characterized by the rule that an operation is activated as soon as all the needed input data is available. Data-driven execution control is characteristic of the dataflow model of computation.

In **Demand-driven execution** the operations will be activated only when their execution is needed to achieve the final result. Demand-driven execution control is typically used in the applicative computational model.

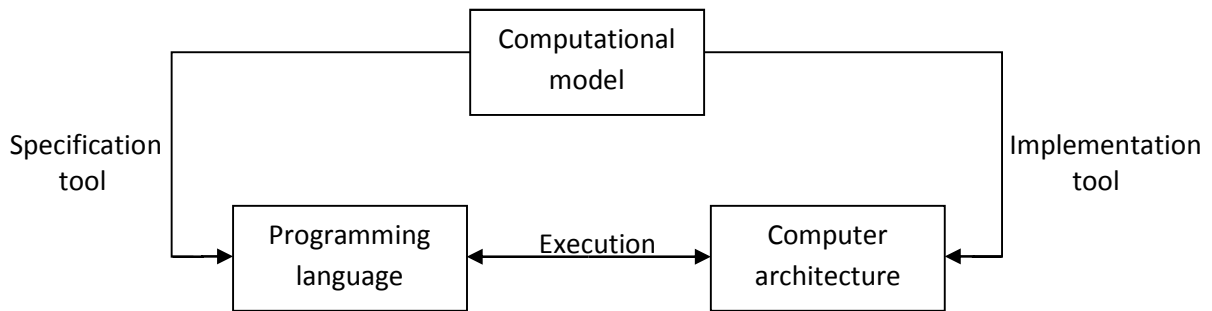
Relationships between the concepts of computational model, Programming language and architecture

The concept of a computational model is a higher level abstraction than the concepts of programming language and computer architecture. A programming language can be thought of as a specification tool making possible the formulation of a computational task whereby a particular computational model is presumed. A computer architecture may be looked upon as a tool to implement a computational model, or to execute a given computational task expressed by means of a programming language, whereby a particular computational model is given.

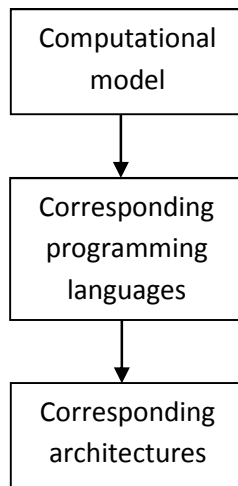
In order to illustrate these relationships we will examine the basic features of the programming languages and computer architectures that correspond to the von Neumann computational model.

A programming language corresponding to this model should allow variables to be declared and their values to be manipulated by means of a proper set of instructions as many times as required during computation. Furthermore, the language should provide control instructions to allow explicit control of the execution sequence. Languages

fulfilling these requirements are called imperative languages. The most widely used languages, such as C, PASCAL, FORTRAN and so on, are all imperative languages.



Interrelationship between the concepts of computational model, programming language and computer architecture.



Typical evolution sequence of a computational model and the corresponding languages and architectures.

Basic computational models – Here, we identify the following six computational models, as –

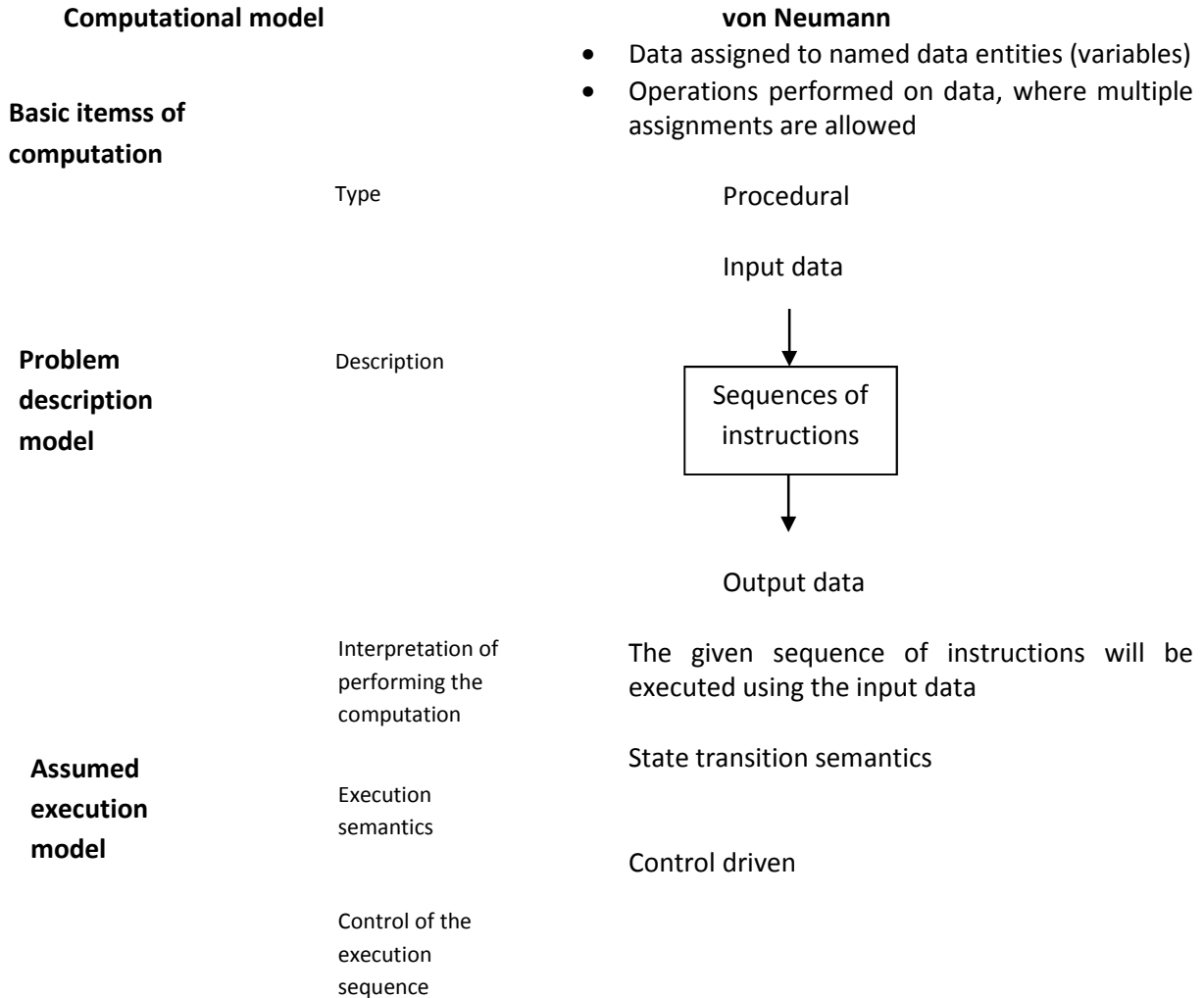
- Turing
- von Neumann
- dataflow
- applicative
- object based
- predicate logic based.

These models are called basic models because they may be declared using a minimal (U;wure) set of abstractions. For each model we specify

- the basic items of computation
- the problem description model
- the execution model

The von Neumann computational model

- **Description of the model** :- We designate the underlying model of von Neumann computers and the corresponding languages as the **von Neumann computational model**. The key features of this oldest and most widely used data-based model may be summarized as follows –



Key features of the von Neumann computation model.

The basic items of computation are data, where computation mean operations performed on data. The data items are identified by names in order to distinguish between different data items used in the same computation. These **named data entities** are called variables in programming languages.

The problem description is procedural, whereas the computational task is specified as a sequence of instructions. Performing the computation means simply that the given sequence of instructions is executed.

The execution of the instructions follows a state transition semantics; that is, as far as execution is concerned the von Neumann model behaves like a finite state machine (FSM).

Key characteristics of the von Neumann model - The main attributes of the model follow from the possibility of multiple assignments of data as well as from the control-driven execution.

Consequences of multiple assignments of data - A consequence of multiple assignments is that the values of the variables may be the actual values of the variables are history sensitive, that is, they are dependent on the actual sequence of the executed instructions.

Consequences of control-driven execution - Because the computational task is specified as an ordered sequence of instructions and the execution sequence of these instructions is control-driven, this model of computation is basically a sequence one. Such an essentially sequential problem description forces sequential processing even in those cases where the problem is inherently parallel.

Related languages - The related languages to von Neumann computational model are called **imperative languages**, emphasizing the possibility of multiple assignments, or sometimes procedural languages, pinpointing the procedural nature of the problem description that they support. Because the procedural style of problem description is used in more than one computational model, in the following we refer to use the term 'imperative languages' to denote the language class corresponding to the von Neumann computational model. Examples of imperative languages are C, Pascal, ALGOL, Fortran, and so on.

Key concepts relating to computational models

1. Granularity -

One of the vital decisions relating to a computational model concerns the types of items that the computation should be performed on and the kinds of operation allowed on them. Possible alternatives are data, objects, sets and so on, along with the declared operations. These items and operations may, however be, of quite different complexity. In order to reflect the complexity of the items of computation, it makes sense to introduce the concept of granularity. Then items of computation can be informally classified as fine-grained or coarse-grained, according to their complexity; in some cases middle-grained items of computation can also be distinguished. Accordingly, as far as von Neumann architectures are concerned, RISC, conventional CISC, HLL and application-oriented architectures may be distinguished as subclasses with increasing granularity.

The concept of granularity was introduced in connection with parallel architectures as well. In this case the granularity refers to the size of the computations that can be executed in parallel without any synchronization or communication. Usually, the parallelism of individual operations is designated as fine-grain parallelism, while parallelism at the thread or process level is called coarse-grained parallelism.

Clearly, the computation/communication ratio is better in coarse-grain systems. On the other hand, the exploitable parallelism is usually much less in coarse-grain systems than in fine-grain parallel systems.

It is useful to introduce the concept of granularity at a higher abstraction level as well, that is, in connection with computation models. Concerning computational models, the granularity will be understood in a more general sense than discussed before. This means that in sequential environments the granularity is interpreted as the complexity of the items of computation, whereas in parallel environments as the size of parallel computations. This interpretation is vague and imprecise, but concerning granularity it yields a unified framework for both architectures and languages.

Architecture class			
Granularity	von Neumann	Dataflow	Reduction
 Low	RISC	Fine grain	Fine grain
	Conventional CISC	(Medium grain)	(Medium grain)
	HLL	Coarse grain	Coarse grain
	Application oriented		
High			

The interpretation of granularity for different fro architecture classes.

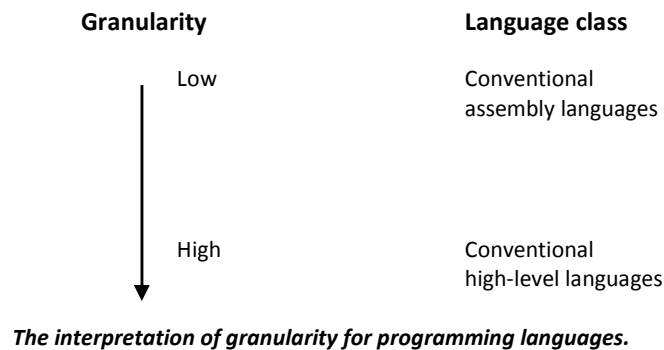
2. Typing

Typing is another concept which is worth introducing at a higher level as it is often used in connection with programming languages. If the concept of typing is introduced at the abstraction level of the computational model, typing of languages and tagging of architectures are closely associated.

In **typed languages** there exists a concept of data type and the language system (compiler or interpreter) may check the consistency of the types used in expressions, function invocations and so on. If the language is **strongly typed**, a type mismatch will cause an error. In **weakly typed** languages, a type mismatch is allowed under given circumstances, namely if the types involved are consistent.

LISP and FP are examples of untyped languages. Pascal, Miranda and HOPE are strongly typed languages, whereas the single assignment language Sisal is weakly typed.

Typed architectures are commonly called tagged. They provide a mechanism for typing the data being stored or processed, by extending the data word by a tag. The tags (usually 3-5 bits long) contain type-identification.



Evolution of computer architecture - As far as the evolution of the concept of computer architecture is concerned, there are four major steps to be emphasized, which will be overviewed as follows –

The term ‘computer architecture’ was coined in 1964 by the ‘chief architects’ of the IBM System/360 in a paper announcing the most successful family of computers ever built. They interpreted computer architecture as ‘the structure of a computer that a machine language programmer must understand to write a correct program for a machine’. Essentially, their interpretation comprises the definition of registers and memory as well as of the instruction set, instruction formats, addressing modes and the actual coding of the instructions excluding implementation and realization. By implementation they understood the actual hardware structure and by realization the logic technology, packaging and interconnections.

Bell and Newell made an important contribution to the concept of computer architecture by introducing a hierarchical, multilevel description (Bell and Newell, 1970). They identify four levels that can be used for describing a computer. These are the electronic circuit level, the logic design level, the programming level and the processor-memory-switch level. The third level refers to the concept of architecture mentioned above. The fourth level is a top-level description of a computer system based on the specification of the basic units like the processor, memory and so on, as well as their interconnections.

The next step in refining the concept of computer architecture was to extend the concept of architecture equally to both the functional specification and the hardware implementation (Sima 1977, Dasgupta 1981).

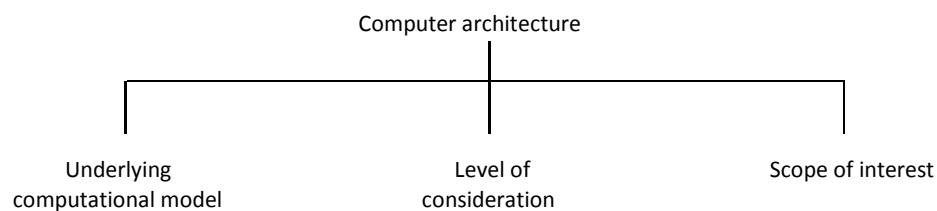
Since the authors just quote also made use of the multilevel description approach, they introduced a ‘two dimensional’ interpretation. Here, one of the ‘dimensions’ is the level of abstraction, in much the same way as introduced by Bell and Newell (1970). The other, orthogonal ‘dimension’ is the scope of interest. Here, they differentiate at each level between a black-box-like functional specification and the description of the

implementation. Correspondingly, they distinguish between ‘logical’ and ‘physical’ architecture (Sima 1977) and endo- and exo- architecture (Dasgupta 1981), respectively.

As pointed out later in this section, the two dimensions considered by these authors, constitute single, hierarchical description framework with increasing levels of abstraction.

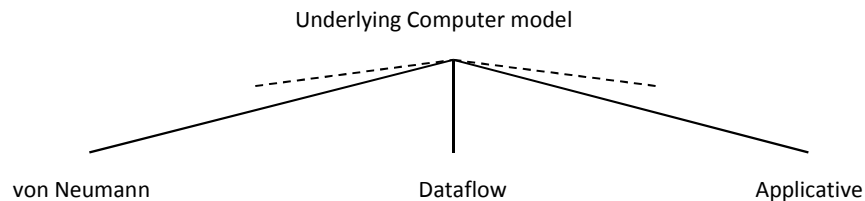
Although this interpretation of the architecture concept is quite useful for a broad range of computers, it assumes the use of the von Neumann model of computation. Therefore, in order to generalize the concept of computer architecture, we shall include in its definition a ‘third dimension’, the model of computation used.

Recent (Latest) interpretation of computer architecture - We interpret the concept of computer architecture at a number of levels of increasing abstraction. At each level the architecture will be described by stating the underlying computational model, the functional specification as well as the actual implementation. Thus, our interpretation covers three aspects; the underlying computational model, the level of consideration and the scope of interest, as shown –



Aspects of interpreting the concept of computer architecture.

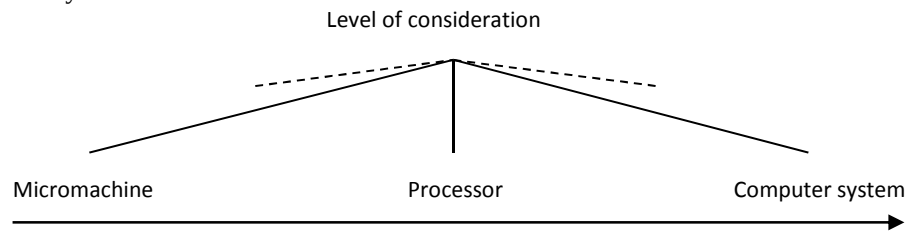
First, we briefly overview the underlying computational model. A couple of years ago the term ‘computer architecture’ was inherently interpreted as a von Neumann architecture. Subsequently, when novel architectures based on a model of computation other than the von Neumann model were introduced, these architectures were properly labeled, for example dataflow architecture, reduction architecture and so on. However, at present it is evident that the concept of computer architecture should include the specification of the underlying computational model as well. Here, we recall the most important models relating to architectures.



Most important computational models related to architectures.

Concerning the level of consideration, there are mainly three levels of interest (in increasing degree of abstraction):

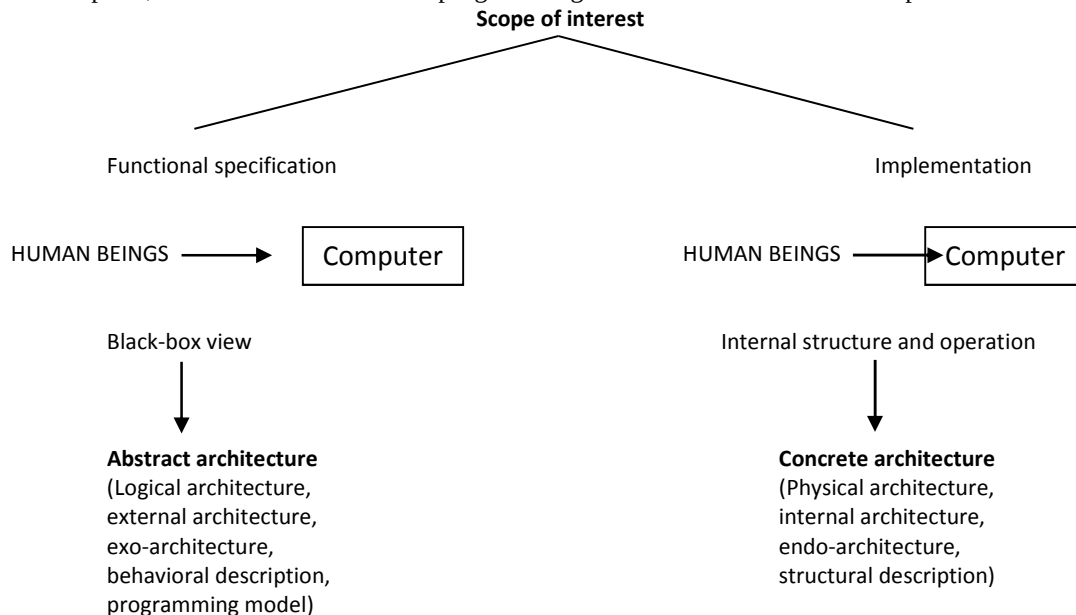
- the micromachine level (in microprogrammed processors),
- the processor level, and
- the computer-system level



Level of consideration in interpreting the concept of computer architecture.

The term ‘architecture’ can be used at each level of consideration with two distinct scopes of interest. If we are interested in the functional specification of a computer, we are dealing with its abstract architecture. Or, if we are interested in its implementation, we are concerned with its concrete architecture. Evidently, the abstract architecture reflects the black-box view whereas the concrete architecture covers the description of the internal structure and operation.

The abstract architecture is also referred to as an exo-architecture, an external or logical architecture, a black-box description, or in certain contexts as a programming model and behavioural description.



Scope of interest in connection with the term architecture.

The abstract architecture is a black-box specification

- either from the programmer’s point of view, or
- from the hardware designer’s point of view.

In the first case we are dealing with the programming model, in the second with the hardware model.

While the programming model is a black-box specification from the programmer’s point of view, the hardware model is viewed as a black-box description for a hardware designer.

Concrete architecture - Concrete architecture is designed by different authors as an endo-architecture, or sometimes as an internal or physical architecture or a structural description. It specifies how a computer is implemented at a particular level of abstraction, whereby a particular computational model is taken for granted. By implementation we mean the internal structure and operation, which may be given by specifying a set of components and their interconnections as well as the possible data- and control- flows.

The concrete architecture can also be considered from two different points of view: logic design or physical design.

The logic design is an abstraction of the physical design and precedes it. Its specification requires:

- the declaration of the logical components used, such as registers, execution units and so on.
- the specification of their interconnections, and
- the specification of the sequence of information transfers, which are initiated by each of the declared function.

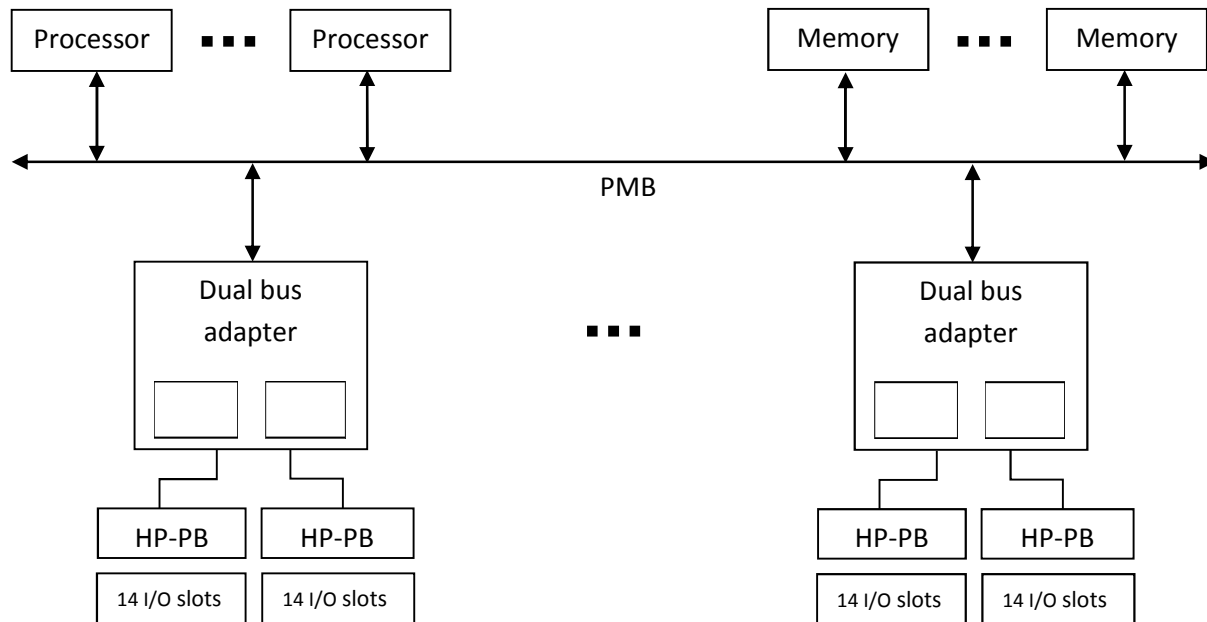
The physical design is based on concrete circuit elements. The specification of a physical design covers:

- the declaration of the circuit elements used, which also includes the specification of signals,
- the specification of their interconnections, and
- the declaration of the initiated signal sequences.

Usually, both the logic and physical design are described in terms of the next lower level of components. For example, the concrete architecture of a processor would be best described in terms of functional units such as register files, execution units, buses and so on.

Computer architecture at different level of abstraction - The most important particular interpretations of the concept of computer architecture are the concrete architecture at the system level, the abstract and concrete architecture at the processor level and the abstract and concrete architecture at the micromachine level.

1. Concrete architecture of computer systems -



PMB: Processor-memory bus

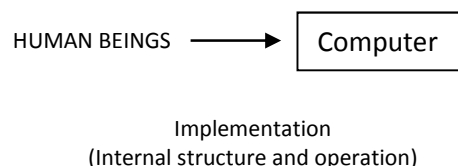
HP-PB: HP precision bus

Concrete architecture at the system level.

At the system level the description of the concrete architecture is based on processor-level building blocks, such as processors, memories, buses and so on. Its description includes the specification of the building blocks, of the interconnections among them and of the whole system.

2. Abstract architecture of processors (architecture)

The abstract architecture of a processor is often referred to as simply the architecture of the processor. It reflects the black-box description of a processor, as shown –



The interpretation of the processor-level abstract architecture.

The black-box description can be considered from either the programmer's or the hardware designer's point of view. Thus when speaking about the processor-level abstract architecture, we are concerned with either the programming model or the hardware model of a particular processor.

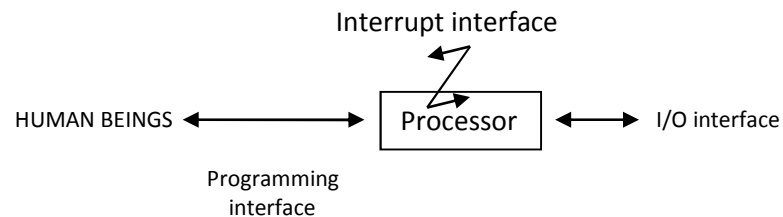
The programming model is equivalent to the description of the machine language. The original definition turns out to be the abstract architecture of the processor, when considered from the programmer's point of view, assuming the von Neumann model of computation.

The programming model, which is given by the machine language, is often described at two different privilege levels: the user level and the system level. Instructions such as I/O instructions and some status manipulation instructions are only available at the system level. For obvious reasons, the programming model is usually declared by the specification of the assembly language rather than by that of the cumbersome machine language.

The programming model is based on the declaration of the available data and status space, expressed in terms of the available memory address space, data registers and status registers. It also includes the specification of the available data and status manipulations. The specification of data manipulations covers:

- the declaration of available data types and the operations on them, and
- for each of the operations, the accessible operand types, the available addressing modes and the actual coding of the instructions.

By contrast, the hardware model of the processor is a description from the hardware designer's point of view. Here, the processor is considered as a building block of a computer system. This can be specified by three interfaces: the programming interface, the interrupt interface and the I/O interface.



The abstract architecture of the processor viewed as a building block.

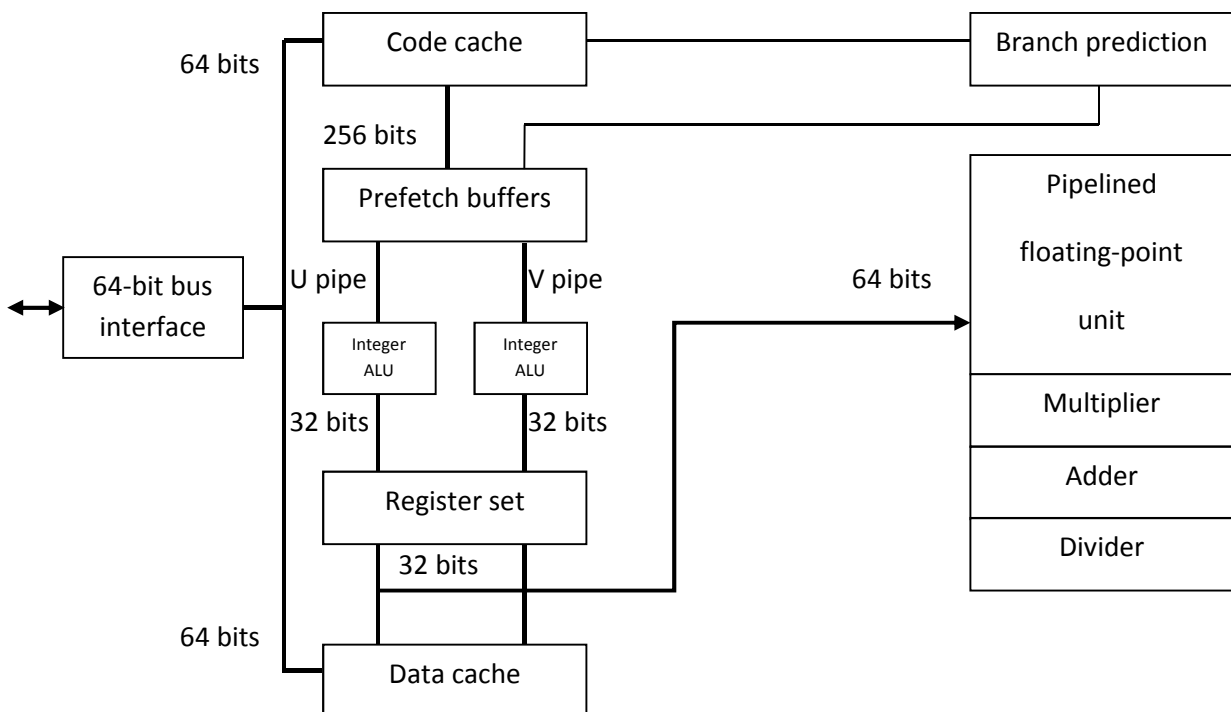
The programming interface is equivalent to the programming model. The other two interfaces specify the behavior of the interrupt system and the I/O system, respectively.

3. Concrete architecture of processors (microarchitecture)

The concrete architecture of a processor is often referred to as the microarchitecture. In publications, the microarchitecture of processors is usually given as a logic design.

Usually, this is informally described by means of a block diagram, through the specification of a set of functional units and their interconnections, and by declaring the operation of the whole processor.

The microarchitecture as a physical design is usually detailed in technical documentation, which is in most cases proprietary information.



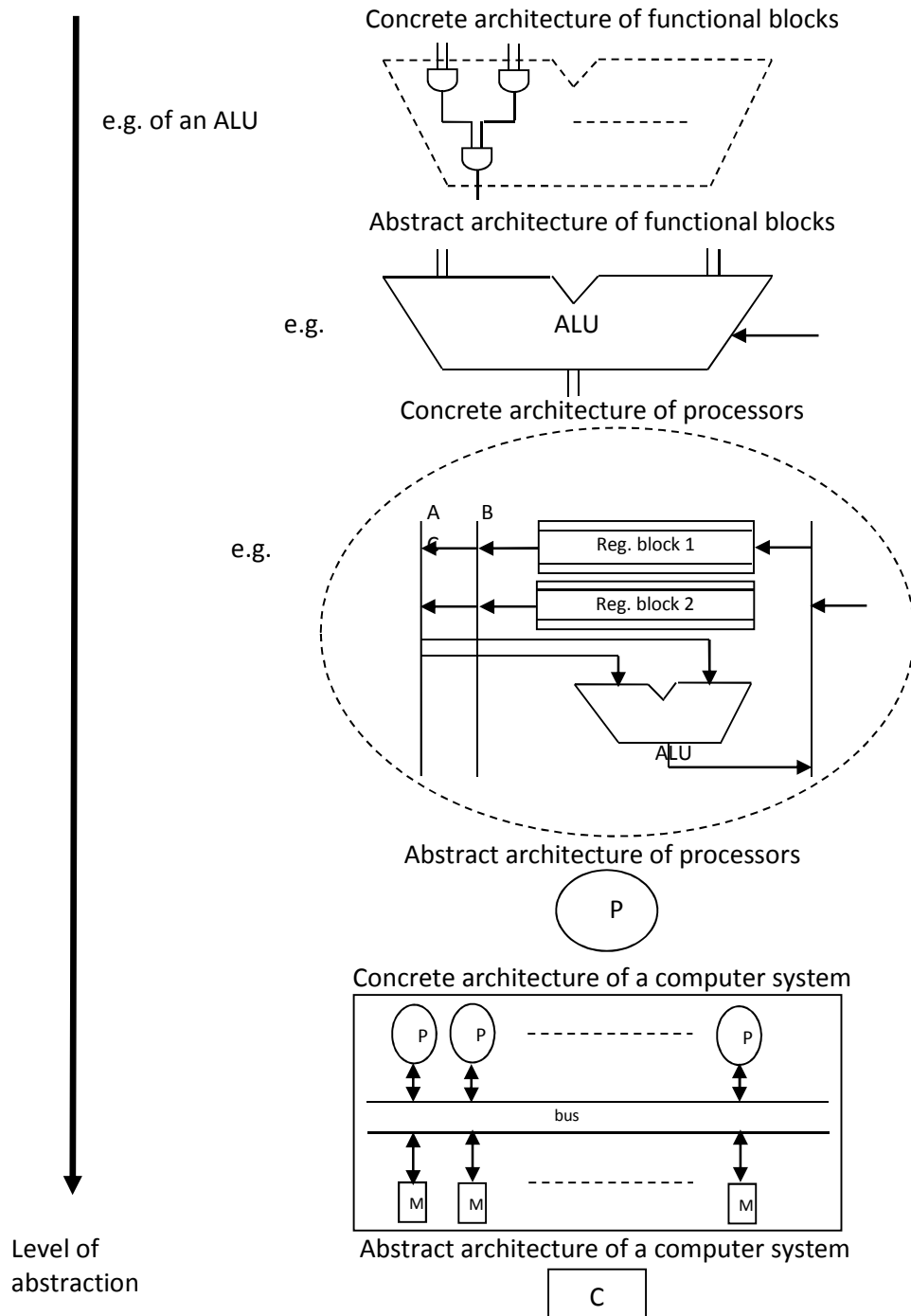
Simplified microarchitecture of the Pentium, given as a logic design.

4. Abstract and concrete architecture at the micromachine level

If the computer is microprogrammed, the micromachine level is also of interest. The abstract architecture at the micromachine level specifies the microprogramming model whereas the concrete architecture at this level describes the internal structure and operation of the micromachine.

The micromachine level was of much interest in connection with sequential microprogrammed computers.

Computer architecture as a multilevel hierarchical framework



Architecture concepts at increasing levels of abstraction.

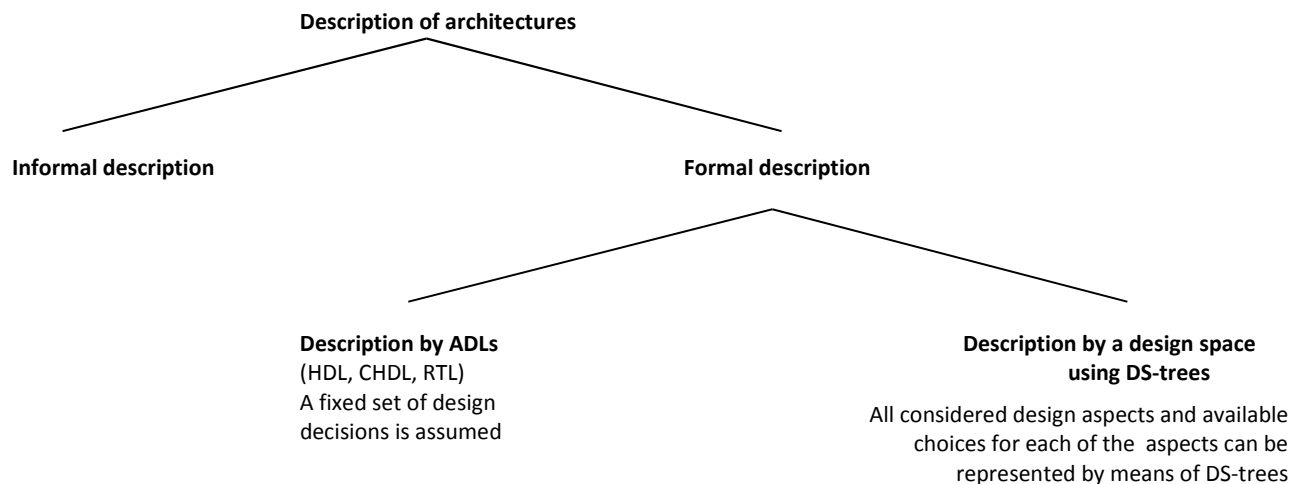
The concrete architecture at a given level is usually described in terms of its components. Thus, the description of the concrete architecture at a given level is based on the abstract architectures of its components. As a consequence, the concrete architecture at a particular level is a description at a higher abstraction level than the corresponding abstract architecture at the next lower level.

Therefore, we can state that the sequence of concepts of concrete and abstract architectures at subsequent levels yields a description framework at consecutively higher levels of abstraction. Consequently, the three-level architecture description scheme discussed, with separate concrete and abstract views at each level, actually provides a six-level hierarchical description scheme. Here, the subsequent levels – the concrete and abstract architecture at the micromachine level, the concrete and abstract architecture at the processor level, and the concrete and abstract architecture at the system level – are levels with an increasing degree of abstraction.

Description of computer architecture - Particular computer architectures are usually described informally, for example in processor handbooks, using a convenient method of presentation. An informal description is also used in most papers introducing a new architecture or in papers comparing existing architectures.

However, a formal description is required if a precise, unambiguous representation is needed, such as for inputting architecture specifications into a CAD package. This may be required in order to perform a verification, simulation or analysis based on the given structural or behavioural description.

The usual method of formally describing an architecture is by using an architecture description language (ADL). Sometimes ADLs are also called hardware description languages (HDLs), or computer hardware description languages (CHDLs). A further concept, which is also used, is register-transfer languages (RTLs). RTL designates a particular type of ADL, which is confined to the processor level. Occasionally, ADLs and HDLs will be distinguished on the basis of the level of abstraction they cover. HDLs are considered as low-level and ADLs as higher-level description languages.



Most important computational models related to architectures.

Concept of program - From the programmer's perspective a program is an ordered set of instructions. On the other hand, from the point of view of an operating system, it is an executable file stored in secondary (auxiliary) memory, typically on a disk. The file P1.EXE is stored in secondary memory and can be accessed through the corresponding directory entry and file description.

Concept of process - In operating system terminology, the notion of process is used in connection with execution instead of the term 'program'. It designates a commission or job, or a quantum of work dealt with as an entity. Consequently, the resources required, such as address space, are typically allocated on a process basis. Each process has a life cycle, which consists of creation, an execution phase and termination. Process creation involves the following four main actions:

- setting up the process description,
- allocating an address space,
- loading the program into the allocated address space, and
- passing the process description to the scheduler.

Usually, operating systems describe a process by means of a description table which we will call the Process Control Block or PCB. A PCB contains all the information relevant to the whole life cycle of a process. It holds basic data such as process identification, owner, process status, description of the allocated address space and so on.

A second major component of process creation is the allocation of address space to a process for execution. There are two approaches: sharing the address space among the created processes (shared memory) or allocating distinct address spaces to each process (per-process address spaces). Subsequently, the executable program file will usually be loaded into the allocated memory space.

Finally, the process thus created is passed to the process scheduler which allocates the processor to the competing processes. The process scheduler manages processes typically by setting up and manipulating queues of PCBs. Thus, after creating a process the scheduler puts the PCB into a ready-to-run processes.

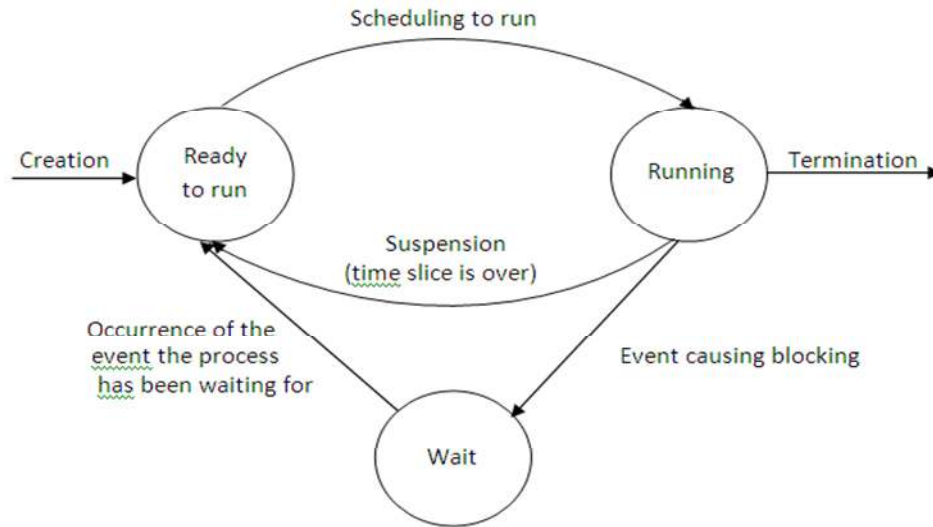
Summing up, process creation essentially consists of setting up the PCB, allocating a shared or a per-process address space to the process, loading the program file and putting the PCB into the ready-to-run queue for scheduling.

The execution phase of a process is under the control of the scheduler. It commences with the creation of a process and lasts until the process is terminated. There are two basic scheduling models used by operating systems, termed the process model and the process-thread mode. They differ essentially in the granularity of the units of work scheduled as an entity.

In the process model scheduling is performed on a per-process basis, that is, the smallest unit of work to be scheduled is a process. The process-thread model is a finer-grained scheduling model, where smaller units of work, called threads, are scheduled as entities.

Process scheduling involves three key concepts: the declaration of distinct process states, the specification of the state transition diagram and the statement of a scheduling policy.

As far as process states are concerned, there are three basic states connected with scheduling: the ready-to-run state, the running state and the wait (or blocked) state. In the ready-to-run state processes are able to run when a processor is allocated for them. In this state they are in execution on the allocated processor. In the wait state they are suspended or blocked waiting for the occurrence of some event before getting ready to run again. When the scheduler selects a process for execution, its state is changed from ready-to-run to running.



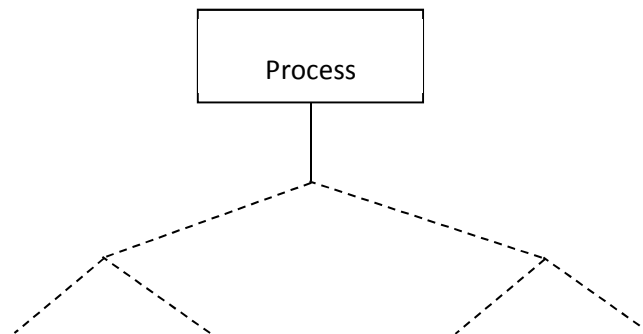
State transition diagram for the basic states.

Finally, a process in the wait can go into the ready-to-run state, if the event it is waiting for has occurred.

Concept of thread

A thread, like a process, is a sequence of instructions. Threads are created within, and belong to, processes. All the threads created within one process share the resources of the process, in particular the address space. Scheduling is performed on a per-thread basis.

Threads have a similar life cycle to the processes and are mainly managed in the same way. Initially each process is created with a single thread. However, threads are usually allowed to create new ones using particular system calls. Then, a thread tree is typically created for each process.



Thread tree.